

Nội dung thực hành:

- Đăng nhập và mã hóa mật khẩu bằng Bcrypt.
- Lưu và quản lý dữ liệu bằng Session.
- Đăng xuất.
- Xây dựng AuthMiddleware, GuestMiddleware.
- Bảo vệ Form bằng CSRF.
- Phân quyền Admin/Staff.
- Tìm hiểu chức năng Remember Me.

Câu A. SỬ DỤNG LẠI PROJECT MiniShop_HoTen	2
Câu B. CHỨC NĂNG ĐĂNG NHẬP	2
Câu C. CHỨC NĂNG ĐĂNG XUẤT	7
Câu D. CHỨC NĂNG MIDDLEWARE	7
Câu E. BẢO VỆ FORM BẰNG CSRF	9
Câu F. CHỨC NĂNG PHÂN QUYỀN CƠ BẢN	12
Câu G. CHỨC NĂNG GHI NHỚ ĐĂNG NHẬP	12
Câu H. TỔNG HỢP CÁC CÂU HỎI CẦN TRẢ LỜI	13
Câu I. ĐĂNG SOURCE CODE THAY ĐỔI LÊN GITHUB.....	13

Câu A. SỬ DỤNG LẠI PROJECT MiniShop_HoTen

Nếu làm trên máy khác, clone phiên bản mới nhất của Project từ GitHub về máy tính bằng lệnh:

- git clone https://github.com/<username>/HoTenSVKhongDau_LTW1.git

Câu B. CHỨC NĂNG ĐĂNG NHẬP

1. Tạo views/admin/login.php

- Không sử dụng layout chung
- Có thể sử dụng Bootstrap để thiết kế giao diện.

```
<html>
<head> ...</head>
<body>
  <div class="container">
    <div class="row justify-content-center mt-5">
      <div class="col-md-5">
        <div class="card shadow">
          <div class="card-body p-4">
            <h3 class="text-center mb-4">Đăng nhập</h3>
            <form action="login.php" method="POST">
              <div class="mb-3">
                <label class="form-label">Tên đăng nhập</label>
                <input type="text" name="username" class="form-control"
                  placeholder="Nhập tên đăng nhập" required>
              </div>
              <div class="mb-3">
                <label class="form-label">Mật khẩu</label>
                <input type="password" name="password" class="form-control"
                  placeholder="Nhập mật khẩu" required>
              </div>
              <div class="mb-3 form-check">
                <input type="checkbox" name="remember" class="form-check-input"
                  id="remember">
                <label class="form-check-label" for="remember"> Ghi nhớ đăng nhập</label>
              </div>
              <div class="d-grid">
                <button type="submit" class="btn btn-primary">Đăng nhập</button>
              </div>
            </form>
          </div>
        </div>
      </div>
    </div>
  </div>
</body>
</html>
```

2. Tạo phương thức findByUsername() trong UserDao

```
public function findByUsername(string $username): ?User
{
    $sql = "SELECT * FROM users
           WHERE username = ?";
    $stmt = $this->conn->prepare($sql);
    $stmt->bind_param("s", $username);
    $stmt->execute();
    $result = $stmt->get_result();
    $row = $result->fetch_assoc();
    if (!$row) {
        return null;
    }
    return new User(
        $row["fullname"],
        $row["username"],
        // .....
    );
}
```

3. Gọi trong login.php

```
$errors = [];
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $username = trim($_POST["username"] ?? "");
    $password = $_POST["password"] ?? "";

    // validate
    if ($username === "") {
        $errors["username"] = "Vui lòng nhập tên đăng nhập.";
    }
    if ($password === "") {
        $errors["password"] = "Vui lòng nhập mật khẩu.";
    }

    // Nếu không có lỗi thì tìm user
    if (empty($errors)) {
        $userDAO = new UserDao();
        $user = $userDAO->findByUsername($username);

        if (!$user) {
            $errors["username"] = "Tên đăng nhập không tồn tại.";
        } elseif (!password_verify($password, $user->password)) {
            $errors["password"] = "Mật khẩu không chính xác.";
        }
    }
}
```

4. Hiện thị lỗi

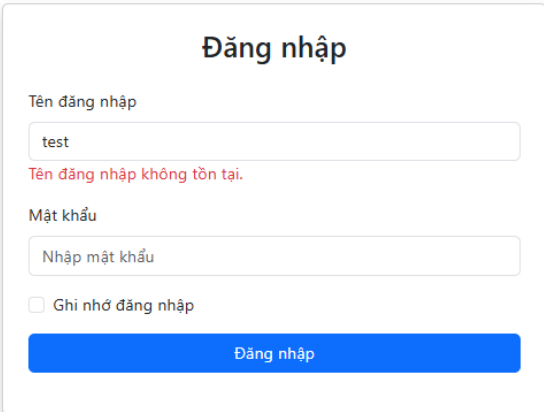
```
<div class="mb-3">
  <label class="form-label">Tên đăng nhập</label>
  <input type="text"
    name="username"
    value="<?= htmlspecialchars($username ?? ") ?>"
    class="form-control">

  <?php if (isset($errors["username"])): ?>
    <div class="text-danger">
      <?= $errors["username"] ?>
    </div>
  <?php endif; ?>
</div>
```

Tìm hiểu thêm về class is-invalid và invalid-feedback trong bootstrap

Hai class này thường được sử dụng để **hiển thị trạng thái lỗi của Form**.

5. Chạy và quan sát kết quả:

Nhập Username và Password không tồn tại trong hệ thống	
Nhập Username tồn tại nhưng Password không đúng	Hiện thị lỗi tương ứng
Để trống Username/Password + Mở DevTools → Inspect . + Xóa thuộc tính required của các ô nhập liệu => Nhấn đăng nhập	Hiện thị lỗi tương ứng
Nhập đúng Username và Password hợp lệ	Không có hiện thị lỗi

6. Commit kết quả vừa làm

7. Lưu User vào Session

- HTTP hoạt động theo nguyên tắc mỗi Request là độc lập.
- Mỗi Request là một lần gửi yêu cầu đến Server. Dữ liệu của Request không được giữ lại ở Request tiếp theo
- **Session** dùng để lưu trữ và duy trì dữ liệu giữa nhiều Request của cùng một phiên làm việc.
- GET/POST → dùng để truyền dữ liệu từ Request này sang Request khác.
- Session → dùng để lưu và duy trì dữ liệu qua nhiều Request mà không cần gửi lại dữ liệu đó trong mỗi Request.

Khởi động session	Đặt ở đầu login.php session_start();
Lưu User vào Session	<code>\$_SESSION["user"] = \$user;</code>
	<pre>// Nếu không có lỗi thì tìm user if (empty(\$errors)) { \$userDAO = new UserDAO(); \$user = \$userDAO->findByUsername(\$username); if (!\$user) { \$errors["username"] = "Tên đăng nhập không tồn tại."; } elseif (!password_verify(\$password, \$user->password)) { \$errors["password"] = "Mật khẩu không chính xác."; } else { \$_SESSION["user"] = \$user; // Chuyển đến Dashboard header("Location: dashboard.php"); exit; } }</pre>

- **Khi sử dụng session:**
 - PHP tạo **Session ID** nếu chưa có.
 - Gửi Session ID về trình duyệt thông qua Cookie, thường tên PHPSESSID
 - Ở các Request tiếp theo, trình duyệt tự gửi PHPSESSID này lên Server.
 - PHP dùng PHPSESSID để tìm lại dữ liệu Session tương ứng.

8. Kiểm tra session

- Sau khi đăng nhập thành công
- Kiểm tra bằng DevTools → Application → Cookies để thấy PHP tạo cookie:

PHPSESSID = xxxxxxxxxxxxxx

Đây là Session ID mà trình duyệt dùng để nhận diện Session.

- Chuyển Qua trang dashboard => Mở lại Cookies và quan sát: PHPSESSID không đổi
- Kiểm tra đọc lại dữ liệu trong session tại dashboard.php

```
session_start();  
var_dump($_SESSION); // xóa dòng này sau khi kiểm tra xong
```
- Chụp màn hình các bước kiểm tra và dán vào file Lab10_capture.docx

9. Hiện thị thông tin người đăng nhập

- Đọc dữ liệu từ session `$_SESSION`
- Hiện thị tên người dùng tại header.php

header.php	<pre><?php \$user = \$_SESSION["user"]; ?> <div class="container-fluid d-flex justify-content-between align-items-center"> <button id="btnMenu" class="btn btn-outline-secondary"> <i class="bi bi-list"></i> </button> <div class="d-flex align-items-center gap-2"> <i class="bi bi-person-circle fs-3"></i> <?= htmlspecialchars(\$user->fullname) ?> </div> </div></pre>
master.php	<pre><?php require_once __DIR__ . '/../../models/User.php'; session_start(); ?> <!DOCTYPE html> <html lang="vi"></pre>

10. Chạy và quan sát kết quả:



11. Commit kết quả vừa thực hiện

Câu C. CHỨC NĂNG ĐĂNG XUẤT

1. Các bước thực hiện:

Tạo file views/admin/logout.php	<pre><?php session_start(); // Khởi động Session session_unset(); // Xóa các biến dữ liệu đang được lưu trong Session. session_destroy(); // hủy dữ liệu Session trên Server. header("Location: login.php"); // Chuyển hướng người dùng về trang đăng nhập. exit; // Dừng chương trình</pre>
Thêm nút Đăng xuất trong header.php	<pre><div class="d-flex align-items-center gap-2"> <i class="bi bi-person-circle fs-3"></i> <?= htmlspecialchars(\$user->fullname) ?> Đăng xuất </div></pre>

2. Kiểm tra sau khi logout:

- Trong file login.php, thêm đoạn code để kiểm tra Session:
 - `var_dump($_SESSION);` // *xóa dòng sau khi kiểm tra xong*
- Vào trang login => nhập thông tin tài khoản tồn tại => đăng nhập thành công
- Nhấn đăng xuất => Quay lại login và kết quả :
 - `array(0) { }`
- Nếu khác kết quả trên kiểm tra lại code trong logout.php

3. Commit kết quả vừa thực hiện

Câu D. CHỨC NĂNG MIDDLEWARE

1. Tạo **AuthMiddleware** kiểm tra đăng nhập để bảo vệ các trang, yêu cầu người dùng phải đăng nhập trước khi được phép truy cập.
 - Nếu chưa đăng nhập, Middleware điều hướng người dùng về trang login.php.
 - Nếu đã đăng nhập, cho phép tiếp tục truy cập trang.
 - Các bước thực hiện:

Tạo file: middleware/AuthMiddleware.php	<pre><?php class AuthMiddleware {</pre>
--	--

static: phương thức thuộc về class, không cần tạo đối tượng để gọi. AuthMiddleware::handle();	<pre>public static function handle() { if (session_status() === PHP_SESSION_NONE) { session_start(); } if (!isset(\$_SESSION["user"])) { header("Location: login.php"); exit; } } }</pre>
Áp dụng Middleware cho nhiều trang +Đặt Middleware ở master.php	<pre><?php require_once __DIR__ . '/../../models/User.php'; require_once __DIR__ . '/../../middleware/AuthMiddleware.php'; session_start(); AuthMiddleware::handle(); ?></pre>

- **Chạy và kiểm tra Middleware:**

- Xóa dữ liệu duyệt web trước khi kiểm tra để đảm bảo không còn trạng thái đăng nhập cũ.
- Chụp kết quả: Chụp màn hình các trường hợp kiểm tra và dán vào file
Lab10_capture.docx.
- Kiểm tra khi chưa đăng nhập:
 - Truy cập URL trang dashboard => hệ thống tự chuyển hướng về login.php
 - Truy cập URL trang sản phẩm => hệ thống tự chuyển hướng về login.php
 - Truy cập URL trang thương hiệu => hệ thống tự chuyển hướng về login.php
- Kiểm tra sau khi đăng nhập:
 - Truy cập URL trang dashboard => hệ thống chuyển hướng về dashboard.php
- Kiểm tra sau khi đăng nhập:
 - Nhấn đăng xuất
 - Truy cập URL trang dashboard => hệ thống chuyển hướng về login.php

- **Commit kết quả vừa thực hiện**

2. Tạo **GuestMiddleware** ngăn người đã đăng nhập truy cập lại trang Login

Tạo file: GuestMiddleware.php (trong thư mục middleware/)	<pre><?php class GuestMiddleware { public static function handle() { if (session_status() === PHP_SESSION_NONE) { session_start(); } if (isset(\$_SESSION["user"])) { header("Location: dashboard.php"); exit; } } }</pre>
Áp dụng GuestMiddleware trong login.php	<pre>require_once __DIR__ . "../dao/UserDAO.php"; require_once __DIR__ . "../middleware/GuestMiddleware.php"; session_start(); GuestMiddleware::handle();</pre>

- Chạy và kiểm tra Middleware:

- Xóa dữ liệu duyệt web trước khi kiểm tra để đảm bảo không còn trạng thái đăng nhập cũ.
- Chưa đăng nhập: Truy cập URL login.php → vẫn ở trang Login và hiển thị Form đăng nhập.
- Đã đăng nhập: Truy cập URL login.php → hệ thống tự động điều hướng về trang dashboard.php.

- Commit kết quả vừa thực hiện

Câu E. BẢO VỆ FORM BẰNG CSRF

1. CSRF là gì:

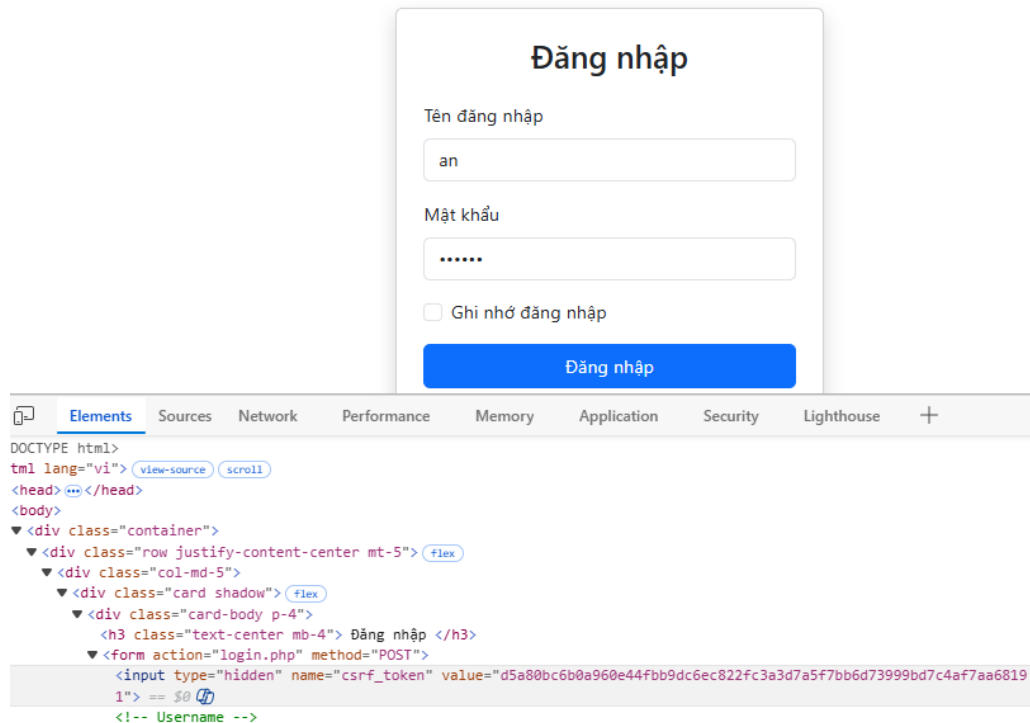
- CSRF (Cross-Site Request Forgery); kiểu tấn công giả mạo Request từ người dùng.
- CSRF Token dùng để kiểm tra Request có hợp lệ hay không
- Một số cách phòng chống CSRF:
 - **CSRF Token:** tạo Token và kiểm tra Token khi nhận Request.
 - **SameSite Cookie:** hạn chế Cookie được gửi trong các Request từ website khác.
 - **Kiểm tra Origin / Referer:** kiểm tra nguồn gửi Request.
 - **Không sử dụng GET cho thao tác thay đổi dữ liệu.**
 - **Các biện pháp khác:** Có thể kết hợp thêm nhiều cơ chế bảo vệ tùy hệ thống.

2. Các bước thực hiện:

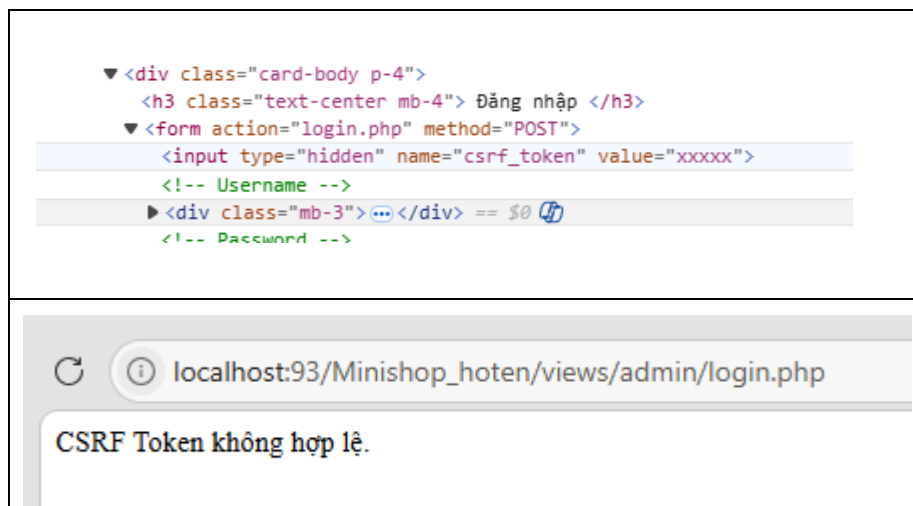
Tạo file: CsrfMiddleware.php (trong thư mục middleware/) + Tạo CSRF Token + Lưu token trong session	<pre><?php class CsrfMiddleware { public static function generateToken() { if (session_status() === PHP_SESSION_NONE) { session_start(); } if (!isset(\$_SESSION["csrf_token"])) { \$_SESSION["csrf_token"] = bin2hex(random_bytes(32)); } } public static function verify() { if (session_status() === PHP_SESSION_NONE) { session_start(); } \$token = \$_POST["csrf_token"] ?? ""; if (!isset(\$_SESSION["csrf_token"]) !hash_equals(\$_SESSION["csrf_token"], \$token)) { die("CSRF Token không hợp lệ."); } } }</pre>
Gọi CsrfMiddleware trong login.php	<pre>require_once __DIR__ . '/../middleware/CsrfMiddleware.php'; // tạo CSRF Token CsrfMiddleware::generateToken(); //.... if (\$_SERVER["REQUEST_METHOD"] === "POST") { // Kiểm tra CSRF Token CsrfMiddleware::verify(); // Validation // // thêm element để giữ csrf <input type="hidden" name="csrf_token" value="<?= htmlspecialchars(\$_SESSION["csrf_token"]) ?>"> // type="hidden" : trường dữ liệu ẩn, người dùng không nhìn thấy trên Form nhưng vẫn được gửi lên Server.</pre>

3. Chạy và kiểm tra Middleware:

- Chưa đăng nhập: Truy cập login.php → Form Login vẫn hiển thị bình thường.
- Kiểm tra CSRF Token: Mở DevTools → Elements, kiểm tra Form có value:
 - `<input type="hidden" name="csrf_token" value="XXXXXX">`



- Token trong Form và Token trong Session phải giống nhau.
- **Sửa lại giá trị token Token:** Dùng DevTools → Nhấn đăng nhập → Request bị từ chối



4. Áp dụng CsrfMiddleware cho các form khác:

- Thực hiện tương tự cho các form thêm, xóa, sửa, các form khác
- Gọi CsrfMiddleware để tạo Token (master.php)
 - CsrfMiddleware::generateToken();
- Thêm CSRF Token vào Form:

```
<input type="hidden" name="csrf_token"
value="<?=
htmlspecialchars($_SESSION["csrf_token"])
?>">
```

- Kiểm tra Token khi nhận Request

```
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    CsrfMiddleware::verify();

    // Xử lý Form
}
```

Câu F. CHỨC NĂNG PHÂN QUYỀN CƠ BẢN

1. Tìm hiểu khái niệm phân quyền (Authorization).
2. Phân biệt đăng nhập (Authentication) và phân quyền (Authorization).
3. Bổ sung thông tin quyền/role cho User, ví dụ: admin, user.
4. Thực hiện kiểm tra quyền sau khi User đăng nhập.
5. Chỉ cho phép User truy cập các chức năng phù hợp với quyền.
6. User không có quyền → từ chối truy cập hoặc điều hướng về trang phù hợp.
7. Kiểm tra ít nhất 2 tài khoản có quyền khác nhau để chứng minh chức năng hoạt động.
8. Chụp kết quả kiểm tra và dán vào Lab10_capture.docx.
9. Commit kết quả sau khi hoàn thành và kiểm tra.

Câu G. CHỨC NĂNG GHI NHỚ ĐĂNG NHẬP

1. Tìm hiểu chức năng Remember Me / Ghi nhớ đăng nhập.
2. Sử dụng Cookie để ghi nhớ trạng thái đăng nhập.
3. Khi người dùng chọn Ghi nhớ đăng nhập, hệ thống có thể tự động đăng nhập lại khi Session không còn.
4. Khi không chọn Ghi nhớ đăng nhập, việc đóng Session phải yêu cầu đăng nhập lại.
5. Đảm bảo không lưu mật khẩu dạng rõ (plain text) vào Cookie.

6. Kiểm tra các trường hợp:
 - Có chọn Ghi nhớ đăng nhập.
 - Không chọn Ghi nhớ đăng nhập.
 - Xóa Cookie và kiểm tra lại.
7. Chụp kết quả kiểm tra và dán vào Lab10_capture.docx.
8. Commit kết quả sau khi hoàn thành và kiểm tra.

Câu H. TỔNG HỢP CÁC CÂU HỎI CẦN TRẢ LỜI

1. Middleware
 - a. AuthMiddleware: Kiểm tra đã đăng nhập chưa
 - b. GuestMiddleware: Kiểm tra đã đăng nhập thì không cho vào Login.
 - c. CsrfMiddleware: Tạo và kiểm tra CSRF Token.
2. Session và Cookie:
 - a. Phân biệt Session và Cookie.
 - b. \$_SESSION["user"] dùng để làm gì?
 - c. GET/POST truyền dữ liệu khác gì với Session?
 - d. Trình bày quá trình Session hoạt động khi người dùng chuyển từ Request này sang Request khác.
3. Đăng nhập, bảo mật và phân quyền:
 - a. password_verify() dùng để làm gì?
 - b. **Bcrypt** là gì? Vì sao mật khẩu trong Database cần được lưu dưới dạng hash bằng **Bcrypt**? Liệt kê một số thuật toán hash mật khẩu khác.
 - c. Vì sao cần kiểm tra Session trước khi cho phép truy cập Admin?
 - d. Khi đăng xuất, cần thực hiện những thao tác nào với Session?
 - e. CSRF Token dùng để làm gì?
 - f. Điều gì xảy ra khi CSRF Token không hợp lệ?
 - g. Phân biệt Authentication và Authorization. Cho ví dụ trong hệ thống MiniShop.

Yêu cầu: Trả lời bằng lời của mình, không sao chép nguyên nội dung từ tài liệu hoặc AI.

Câu I. ĐẨY SOURCE CODE THAY ĐỔI LÊN GITHUB

- Mở terminal tại thư mục chứa project
- Thêm source vào Git, dùng lệnh : git add .
- Commit source: git commit -m "MiniProject - X"
- Đẩy source lên GitHub: git push origin main
- Kiểm tra lại trên GitHub: source code đã được cập nhập